subprocess 학습 노트

날짜: 2026-05-22 복습 예정: D+1, D+3, D+7

한 줄 요약

Python 코드 안에서 터미널 명령어를 실행하는 방법 — `pylint src/auth.py`를 코드로 실행하고 결과를 받아오는 것.

---

1. 비유

터미널에서 직접 입력:

```
pylint src/auth.py --output-format=json
```

Python 코드로 같은 작업:

```
import subprocess
result = subprocess.run(["pylint", "src/auth.py", "--output-format=json"],
capture_output=True, text=True)
print(result.stdout)  # 터미널 출력과 동일한 결과
```

subprocess = Python이 터미널 대신 명령어를 실행해주는 도구.

---

2. 개념

`subprocess.run()`: 외부 프로그램 실행 + 결과 반환

- `capture_output=True`: stdout, stderr를 캡처 (화면에 출력 안 됨)
- `text=True`: 결과를 bytes가 아닌 str로 반환
- `returncode`: 0 = 성공, 0 외 = 오류/이슈 발견

---

3. 왜 쓰나

- 없으면: Pylint/Bandit을 Python API로 직접 호출 → 각 도구마다 다른 API 학습 필요, 버전 의존성 높음
- 있으면: CLI 명령어로 실행 → 도구 교체가 쉽고, CLI 옵션 그대로 사용 가능

Dev 2에서: Pylint, radon, Bandit, Semgrep, pip-audit 모두 subprocess로 실행

---

4. 코드 (Dev 2)

```
# dev2/tools/static/pylint_tool.py
import subprocess, json

def run_pylint(files: list[str]) -> list[dict]:
    if not files:
        return []

    result = subprocess.run(
        ["pylint"] + files + ["--output-format=json"],
        capture_output=True,   # stdout/stderr 캡처
        text=True,             # str 반환 (bytes 아님)
        timeout=30,            # 30초 초과 시 TimeoutExpired 예외
    )

    # Pylint는 이슈가 있어도 returncode != 0 → 이슈 있음 = 정상
    # stdout이 비어있으면 이슈 없음
    if not result.stdout.strip():
        return []

    issues = json.loads(result.stdout)
    return [
        {
            "severity": _map_severity(i["type"]),
            "line": i["line"],
            "file": i["path"],
            "message": i["message"],
            "rule_id": f"SA-{i['symbol'].upper()}",
            "suggestion": "",
            "similar_issues": [],
        }
        for i in issues
    ]

def _map_severity(pylint_type: str) -> str:
    return {"error": "Critical", "warning": "Major", "convention": "Minor",
"refactor": "Minor"}.get(pylint_type, "Minor")
```

주요 패턴 비교:

```
# 기본: 결과 캡처
result = subprocess.run(["pylint", "src/"], capture_output=True, text=True)
result.stdout   # 표준 출력 (JSON 결과)
result.stderr   # 에러 출력
result.returncode  # 종료 코드

# 타임아웃 처리
try:
    result = subprocess.run(["bandit", "-r", "src/"], capture_output=True,
```

```
text=True, timeout=30)
except subprocess.TimeoutExpired:
    return []  # 타임아웃 시 빈 결과 반환

# shell=True는 사용 금지 (보안 위험 — Shell Injection)
subprocess.run("pylint " + user_input, shell=True)   # ❌ 위험
subprocess.run(["pylint", user_input], shell=False)   # ✅ 안전
```

복습 퀴즈

1. Q: `capture_output=True`를 쓰는 이유는? A: (터미널에 출력 대신 result.stdout으로 받아서 Python에서 처리하기 위해)
2. Q: `shell=True`를 쓰면 안 되는 이유는? A: (Shell Injection 보안 취약점. 사용자 입력이 명령어에 포함되면 악의적인 명령 실행 가능)
3. Q: Dev 2에서 subprocess를 쓰는 이유는? A: (Pylint/Bandit 등 CLI 도구를 Python 내에서 실행하고 JSON 결과를 받아 파싱하기 위해)